Function-Preserving Width Splitting of Pre-trained LLMs: Exact Expansion, a Hidden Learning-Rate Change, and the Limits of Symmetry-Breaking Noise

Anonymous ACL submission

Abstract

Growing a pre-trained large language model (LLM) in place, instead of training a larger one from scratch, needs an expansion that preserves the model's function and a way to make the added parameters useful. We study **Discriminative Parameter Splitting for LLMs (DPS-LLM)**, which doubles the intermediate width of every SwiGLU feed-forward block by duplicating the gate and up projections and the $\frac{1}{2}$-scaled down projection, and breaks the symmetry of the copies with a polarity-preserving, anti-symmetric perturbation adapted from LSTM acoustic models. We prove that the split is exactly function-preserving and that the anti-symmetric pairing cancels first-order output drift, and confirm both on seven open-weight models from 360M to 1.7B parameters: in float32, the DPS split changes validation perplexity by at most 1.3×10^{-6}, two orders of magnitude less than Gaussian noise of the same size. Its effect on fine-tuning is not what growth methods assume. We prove that under Adam an exact split trains like its seed with a doubled learning rate on the down projection, and a matched control, the unexpanded model with that learning rate, reproduces the split's perplexity reduction on every model and dataset, within 0.18 points in 14 of 16 settings and within seed noise in the other two. The perturbation does not break the symmetry: in float32 the distance between paired neurons stays constant during fine-tuning, and DPS-LLM, Gaussian noise and exact duplication are statistically indistinguishable. At each model's best learning rate, the split model, 54–75% larger, is ahead of its seed on only three of eight settings. We release code, notebooks and all per-seed results, and argue that evaluations of model growth need matched learning-rate controls.

1 Introduction

Transformer LLMs (Vaswani et al., 2017; Touvron et al., 2023; Yang et al., 2024) are almost always trained under a *static* paradigm: every dimension of the network is fixed *a priori*, the weights are drawn from a random initialization (Glorot and Bengio, 2010; He et al., 2015), and the model is trained end-to-end. Scaling laws (Kaplan et al., 2020; Hoffmann et al., 2022) make the cost of this paradigm explicit: a larger model must be trained from scratch, spending much of its early budget on features a smaller model already possesses. The same rigidity appears at fine-tuning time. When a compact model saturates its parametric capacity on a domain-specific task, practitioners must either accept the plateau or restart with a larger architecture.

Progressive growth offers an alternative: begin with a compact *seed* model and enlarge it during training (Chen et al., 2016a; Gong et al., 2019; Chen et al., 2022; Wang et al., 2023). Two requirements make growth practical for modern LLMs. First, the expansion must be *function-preserving*, so that the grown model neither loses the seed's knowledge nor suffers a loss spike. Second, the duplicated parameters must be *distinguishable*; otherwise identical copies receive identical gradients and the added capacity is never used (Chen et al., 2016a). Existing work on Transformers has addressed these requirements for pre-LayerNorm blocks with ReLU or GELU feed-forward layers (Chen et al., 2022; Wang et al., 2024), but gated SwiGLU blocks (Shazeer, 2020), which are now standard in LLaMA, Mistral, Gemma, Qwen and SmolLM, multiply two projections element-wise, so naive duplication doubles the block's output.

We adapt *Discriminative Parameter Splitting* (Irfan, 2021), originally developed for growing LSTM acoustic models, to SwiGLU Transformers, and ask whether it delivers on both requirements when a pre-trained LLM is fine-tuned. Our contributions are:

- **An exact function-preserving SwiGLU split**

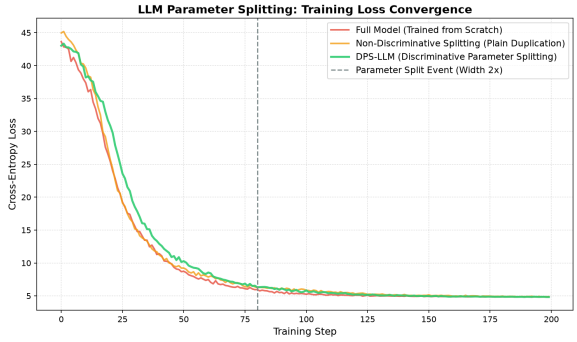


Figure 1: **Controlled split on a synthetic language-modelling task.** A two-layer SwiGLU Transformer with $d_{ffn} = 64$ is trained for 80 steps and then split to $d_{ffn} = 128$ (dashed line). Both split variants continue without a loss spike, whereas the $d_{ffn} = 128$ model trained from scratch spends its first 80 steps re-learning what the seed already knows. DPS-LLM and plain duplication converge to the same loss as the full model by step 200.

with drift-cancelling noise. Duplicating the rows of W_{gate} and W_{up} and the columns of $\frac{1}{2}W_{down}$ doubles d_{ffn} with provably identical output (Theorem 1), and a polarity-signed, anti-symmetric perturbation cancels first-order drift (Proposition 2). Both hold on real checkpoints.

- **A split is a learning-rate change.** Under Adam, an exact split trains like its seed with a doubled learning rate on W_{down} (Proposition 3). We introduce the matched control this implies and show that it reproduces the split's gains.

- **A controlled benchmark.** On seven open-weight models from four families (360M–1.7B), two datasets, five conditions, three seeds, per-model learning-rate sweeps and ablations, we find no gain from added capacity and no symmetry breaking during fine-tuning.

- **A precision pitfall.** In bfloat16 the default perturbation is rounded away entirely; we give a stochastic-rounding protocol under which it survives.

We also describe, and release code for, two further regimes of the same operator—dense-to-Mixture-of-Experts (MoE) upcycling and LoRA rank doubling—which we leave for empirical evaluation in future work (Appendix A).

2 Related Work

Function-preserving growth. Net2Net (Chen et al., 2016a) introduced *Net2WiderNet* and *Net2DeeperNet*, which widen or deepen a network without changing its function, and noted that a small amount of noise is needed to break the symmetry of copied units. Network morphism (Wei et al., 2016) generalised the idea to arbitrary non-linear morphisms. For Transformers, progressive stacking (Gong et al., 2019) and staged training (Shen et al., 2022) grow depth and width of BERT-style encoders; bert2BERT (Chen et al., 2022) extends Net2Net to attention blocks with function-preserving and advanced knowledge initialisation; LiGO (Wang et al., 2023) learns a linear growth operator from the small model; MSG (Yao et al., 2024) and LEMON (Wang et al., 2024) obtain lossless expansion for pre-LayerNorm blocks by masking or by an explicit average-and-cancel construction. Depth up-scaling (Kim et al., 2024) and HyperCloning (Samragh et al., 2024) apply the same principle to decoder-only LLMs, the latter by tiling weights of a small model into a wider one. DPS-LLM belongs to this family but targets the gated SwiGLU block specifically, where the $\frac{1}{2}$ factor must be placed on the down projection to preserve function.

Neuron splitting. Splitting steepest descent (Liu et al., 2019), Firefly descent (Wu et al., 2020) and GradMax (Evci et al., 2022) choose *which* neurons to split and *how* to perturb the copies using second-order or gradient information. These methods are principled but require extra computation per split; DPS-LLM instead splits every neuron and uses a closed-form, gradient-free perturbation whose only statistic is the standard deviation of the weight matrix.

Mixture-of-Experts upcycling. Sparse upcycling (Komatsuzaki et al., 2023) initialises MoE experts (Shazeer et al., 2017; Lepikhin et al., 2021; Fedus et al., 2022) by cloning a dense MLP, and He et al. (2024) study this recipe at scale; Mixtral (Jiang et al., 2024) is a widely used dense-to-sparse architecture. Cloned experts are initially identical, so the same symmetry problem arises; our anti-symmetric noise is directly applicable (Appendix A).

Adapter rank growth. LoRA (Hu et al., 2022) fixes the adapter rank in advance; AdaLoRA (Zhang et al., 2023) and DyLoRA (Valipour et al.,

2023) adapt the rank during training. Rank doubling by parameter splitting (Appendix A) is a complementary, function-preserving way to increase adapter capacity.

Discriminative splitting in speech. Irfan (2021) grew LSTM-RNN acoustic models by duplicating rows and columns of weight matrices and injecting a polarity-preserving perturbation, reporting faster convergence, less over-training and a 1.2% absolute word-error-rate reduction over Xavier initialisation on a Kaldi/VoxForge recipe. This paper transfers that recipe to Transformers and adds the invariance scaling and the anti-symmetric pairing.

3 Method

3.1 Function-preserving SwiGLU width splitting

Modern decoder blocks (Shazeer, 2020; Touvron et al., 2023; Yang et al., 2024) use a gated feed-forward network

$$\mathrm{FFN}(x) = W_{down}\left(\mathrm{SiLU}(W_{gate}x) \odot W_{up}x\right), \tag{1}$$

with $W_{gate}, W_{up} \in \mathbb{R}^{d_{ffn}\times d_{model}}$, $W_{down} \in \mathbb{R}^{d_{model}\times d_{ffn}}$, and $\mathrm{SiLU}(z) = z\,\sigma(z)$ (Hendrycks and Gimpel, 2016; Elfwing et al., 2018; Ramachandran et al., 2017). Let $h(x) = \mathrm{SiLU}(W_{gate}x) \odot W_{up}x \in \mathbb{R}^{d_{ffn}}$.

To grow $d_{ffn} \to 2d_{ffn}$ we apply *rows extension* to the two incoming projections and *columns extension* with a $\frac{1}{2}$ factor to the outgoing projection:

$$W'_{gate} = \begin{bmatrix} W_{gate} + \Delta^{(1)}_{gate} \\ W_{gate} + \Delta^{(2)}_{gate} \end{bmatrix}, \tag{2}$$

$$W'_{up} = \begin{bmatrix} W_{up} + \Delta^{(1)}_{up} \\ W_{up} + \Delta^{(2)}_{up} \end{bmatrix}, \tag{3}$$

$$W'_{down} = \tfrac{1}{2}\begin{bmatrix} W_{down} & W_{down} \end{bmatrix} + \begin{bmatrix} \Delta^{(1)}_{down} & \Delta^{(2)}_{down} \end{bmatrix}. \tag{4}$$

Theorem 1 (Functional invariance)**.** *With $\Delta = 0$, $\mathrm{FFN}'(x) = \mathrm{FFN}(x)$ for every input x.*

Proof. The stacked gate and up projections produce $[h(x);\, h(x)]$, so $\mathrm{FFN}'(x) = \frac{1}{2}W_{down}h(x) + \frac{1}{2}W_{down}h(x) = W_{down}h(x)$. □

Because every block is invariant and the residual stream, attention and normalisation layers are untouched, the whole model is invariant; the loss on any batch is unchanged in exact arithmetic. Without the $\frac{1}{2}$ factor the block's output would double, which is the loss spike observed when SwiGLU networks are naively widened.

3.2 Calculated Anti-Symmetric Noise

If $\Delta = 0$, the two copies of every neuron are identical and receive identical gradients for the rest of training, so the expanded model has no more effective capacity than the seed (Chen et al., 2016a). Isotropic Gaussian noise breaks the tie but is sign-agnostic: it flips the polarity of small weights, which Irfan (2021) found harmful for pre-trained LSTMs. We instead use a perturbation whose sign is dictated by the weight it perturbs. For a weight matrix W with element-wise standard deviation σ_W, draw $u \sim \mathcal{U}(0,1)$ and $n \sim \mathcal{N}(0,1)$ independently per element and set

$$f(W) = \left|(u - \tfrac{1}{2}) \cdot C\,\sigma_W \cdot n\right|, \tag{5}$$

$$\delta(W) = \mathrm{sign}(W) \odot f(W), \tag{6}$$

$$\Delta^{(1)} = +\delta(W), \qquad \Delta^{(2)} = -\delta(W). \tag{7}$$

The modulation constant $C = 10^{-4}$ is inherited from the original speech recipe; (5)–(6) are its Equations 5.7–5.8 with the noise scaled by σ_W so that the same C transfers across layers of different magnitude. Equation (7) is new: the two copies receive *opposite* perturbations. For W_{down} the noise is computed on the already-halved matrix.

Proposition 1 (Sign preservation)**.** *Copy 1 satisfies $|w + \delta(w)| = |w| + f(w) > |w|$ for every weight, so no sign is flipped. Copy 2 satisfies $|w - \delta(w)| = \big||w| - f(w)\big|$ and flips sign only if $|w| < f(w)$.*

Since $f(w) \le \frac{1}{2}C\sigma_W|n|$, a flip requires $|w| < \frac{1}{2}\cdot 10^{-4}\sigma_W|n|$; for approximately Gaussian weights this is a fraction on the order of 10^{-4} of the matrix or less, and these weights carry negligible signal. In our runs the measured sign-flip fraction is 8–9 $\times\, 10^{-6}$ (Section 4.2).

Proposition 2 (First-order drift cancellation)**.** *With the pairing in (7), $\mathrm{FFN}'(x) = \mathrm{FFN}(x) + O(\|\delta\|^2)$.*

Proof. Write $h_1(x)$ and $h_2(x)$ for the hidden activations of the two copies. A first-order expansion in δ gives $h_1 = h + J\delta x + O(\delta^2)$ and $h_2 = h - J\delta x + O(\delta^2)$ for the same Jacobian J, so $\frac{1}{2}W_{down}(h_1 + h_2) = W_{down}h + O(\delta^2)$. The down-projection perturbation contributes $\delta_{down}h_1 - \delta_{down}h_2 = \delta_{down}(h_1 - h_2) = O(\delta^2)$. □

The perturbation therefore does two things at once: it leaves the output unchanged to first order, and it makes $\nabla_{W^{(1)}}\mathcal{L} \neq \nabla_{W^{(2)}}\mathcal{L}$ from the

very first update, because the two copies now see different pre-activations. Both statements assume that the perturbation is representable in the weight format. bfloat16 keeps 8 significant bits, so a perturbation of relative size 10^{-4} is rounded away for all but the smallest weights; Section 4.2 shows that this is what happens in practice.

3.3 An exact split changes the effective learning rate

Function preservation holds at the moment of the split, but not along the optimisation trajectory. With Adam-type optimisers (Kingma and Ba, 2015; Loshchilov and Hutter, 2019), the split also changes the effective step size of the block.

Proposition 3 (Split as learning-rate doubling)**.** *Let the split be exact ($\Delta = 0$) and let both models be trained with Adam at learning rate η. Then, up to Adam's ϵ, the copies stay identical and the split model follows the trajectory of the unsplit model trained with learning rate η on W_{gate}, W_{up} and 2η on W_{down}.*

Proof sketch. While the copies are identical, each copy of a row of W_{gate} or W_{up} receives exactly half the gradient of the original row, because it feeds a down-projection column scaled by $\frac{1}{2}$; Adam's update is invariant to a constant rescaling of the gradient, so both copies take the step the original would take, and stay identical. Each copy of a column of W_{down} receives the full gradient of the original column, because it sees the same activation; both copies take the original's step, and the effective matrix $W^{(1)}_{down} + W^{(2)}_{down}$ moves twice as far. □

The equivalence is exact in exact arithmetic without gradient clipping and with decoupled weight decay ignored; global-norm clipping, which rescales all gradients jointly, and finite precision make it approximate. On a toy SwiGLU block trained with AdamW in float32 for 100 steps, the losses of the exact split and of the unsplit block with a $2\times$ learning rate on W_{down} differ by at most 7×10^{-6}, against 2.4×10^{-2} from the unsplit block at the original learning rate. Comparisons between a split model and its seed at one shared learning rate therefore conflate added capacity with a larger step on W_{down}; Section 4.2 confirms the equivalence on real checkpoints.

3.4 Applying the operator to a pre-trained LLM

The split is applied in place to every decoder layer of a Hugging Face checkpoint (Wolf et al., 2020): the three linear layers are replaced by their expanded versions and `intermediate_size` is doubled in the configuration. No other tensors change. For Qwen2.5-0.5B this grows d_{ffn} from 4864 to 9728 in all 24 layers (+313.8M parameters); for SmolLM-360M from 2560 to 5120 in all 32 layers (+236.0M).

4 Experiments

4.1 Setup

Models and precision. We use seven instruction-tuned open-weight checkpoints with SwiGLU blocks of the form (1): SmolLM-360M, SmolLM2-360M and SmolLM2-1.7B (Allal et al., 2024, 2025), Qwen2.5-0.5B and Qwen2.5-1.5B (Yang et al., 2024), TinyLlama-1.1B (Zhang et al., 2024), and OLMo-2-1B (Team OLMo et al., 2024). The three models below 1B parameters are fine-tuned entirely in float32, where the $C{=}10^{-4}$ perturbation is representable. Models of 1B parameters and more are fine-tuned in bfloat16 to stay within the memory of two 16 GB GPUs after splitting, with bfloat16 weights, an AdamW update computed in float32 and written back with stochastic rounding (Gupta et al., 2015; Zamirai et al., 2020) (so updates smaller than half a unit in the last place are applied in expectation), and $C{=}10^{-2}$, because the split itself is rounded to nearest and a $C{=}10^{-4}$ perturbation would survive in only 0.6% of entries (Section 4.2). To compare the two protocols on one checkpoint, SmolLM-360M is also run under the bfloat16 protocol (marked †).

Data. GSM8K (Cobbe et al., 2021) and Alpaca-Cleaned (Taori et al., 2023; yahma, 2023), rendered with fixed templates (Appendix B). For GSM8K we fine-tune on 256 examples drawn from the training split and measure perplexity on the first 100 test problems. For Alpaca-Cleaned, which has no test split, a fixed permutation of the training split provides 256 fine-tuning and 100 disjoint validation examples. GSM8K accuracy is exact match of the final number, generated greedily (at most 256 new tokens) for the first 50 test problems, before fine-tuning once per model and after fine-tuning for every run.

Table 1: **Relative validation-perplexity reduction (%) after fine-tuning**, mean$\pm$std over three seeds (higher is better; bold: best mean per row). *Base, 2× lr W_{down}* is the unexpanded model trained with a doubled learning rate on the down projection, the matched control of Proposition 3. *DPS − ctrl*: mean paired difference between DPS-LLM and the control, with the paired t-test p-value. Models without a mark are fine-tuned in float32 with $C{=}10^{-4}$; †: bfloat16 with stochastic rounding and $C{=}10^{-2}$.

Model	Data	Base	Base, 2× lr W_{down}	Naive split	Gaussian split	DPS-LLM	DPS − ctrl (p)
SmolLM-360M	GSM8K	28.58±0.30	31.04±0.32	**31.15±0.30**	**31.15±0.30**	**31.15±0.30**	+0.11 (0.04)
SmolLM2-360M	GSM8K	21.51±0.39	22.28±0.32	**22.29±0.32**	**22.29±0.32**	**22.29±0.32**	+0.01 (0.12)
Qwen2.5-0.5B	GSM8K	16.12±3.35	**16.75±2.66**	16.58±2.86	16.60±2.83	16.58±2.86	−0.17 (0.50)
TinyLlama-1.1B†	GSM8K	**49.55±0.21**	47.88±0.45	47.86±0.38	47.85±0.46	47.79±0.43	−0.09 (0.41)
OLMo-2-1B†	GSM8K	**49.88±1.00**	48.83±1.04	48.82±1.00	48.85±1.01	48.89±0.95	+0.06 (0.39)
Qwen2.5-1.5B†	GSM8K	17.20±2.41	17.79±1.77	17.62±1.40	17.12±2.10	**17.83±2.03**	+0.03 (0.90)
SmolLM2-1.7B†	GSM8K	0.55±1.41	**0.82±0.67**	0.46±0.55	0.79±0.99	0.40±0.42	−0.42 (0.19)
SmolLM-360M†	GSM8K	28.57±0.33	31.03±0.31	31.12±0.27	31.14±0.29	**31.16±0.29**	+0.13 (0.07)
SmolLM-360M	Alpaca	29.02±0.08	32.55±0.10	**32.58±0.09**	**32.58±0.09**	**32.58±0.09**	+0.03 (0.14)
SmolLM2-360M	Alpaca	30.27±0.23	**30.91±0.28**	**30.91±0.29**	**30.91±0.29**	**30.91±0.29**	+0.00 (0.62)
Qwen2.5-0.5B	Alpaca	**18.56±0.73**	11.47±2.07	11.47±1.95	11.47±1.95	11.47±1.95	+0.00 (0.96)
TinyLlama-1.1B†	Alpaca	**36.51±0.08**	33.07±0.67	33.05±0.65	33.04±0.63	33.04±0.63	−0.03 (0.25)
OLMo-2-1B†	Alpaca	**44.22±0.79**	41.89±0.85	41.88±0.87	41.81±0.92	41.82±0.97	−0.07 (0.42)
Qwen2.5-1.5B†	Alpaca	**19.71±0.34**	15.50±0.78	15.68±0.40	15.54±0.87	15.43±0.88	−0.07 (0.55)
SmolLM2-1.7B†	Alpaca	31.34±0.25	31.42±0.37	**31.45±0.37**	**31.45±0.35**	31.41±0.45	−0.01 (0.78)
SmolLM-360M†	Alpaca	28.92±0.14	32.49±0.10	32.42±0.10	**32.63±0.06**	32.47±0.15	−0.02 (0.61)

Conditions. (1) *Base*: the unmodified checkpoint. (2) *Base, 2× lr W_{down}*: the unmodified checkpoint with a doubled learning rate on every down projection, the matched control of Proposition 3. (3) *Naive split*: Eqs. (2)–(4) with $\Delta{=}0$. (4) *Gaussian split*: i.i.d. Gaussian noise drawn independently for each copy, as in Net2Net (Chen et al., 2016a), with standard deviation $C\sigma_W/\sqrt{12}$ so that its RMS matches the calculated noise. (5) *DPS-LLM*: Calculated Anti-Symmetric Noise.

Fine-tuning and evaluation. All conditions share one recipe (Table 2); only the SwiGLU parameters are trained, so the comparison isolates the expanded block. Validation perplexity (Jelinek et al., 1977) is exp of the mean per-sequence cross-entropy, measured before the first and after the last update. Every condition is run with seeds 42, 43 and 44, which set the noise draw and the order of the training examples; we report mean±std and paired t-tests across seeds. Every 25 steps we record the mean relative distance $\|a-b\|/\frac{1}{2}(\|a\|+\|b\|)$ between the two copies of every split neuron (rows of W_{gate}, W_{up}; columns of W_{down}). A *learning-rate sweep* runs Base and DPS-LLM at $\{1,2,5,10\}\times10^{-5}$ on GSM8K (seed 42), and *ablations* on SmolLM-360M/GSM8K (float32, seed 42, 50 steps) vary the noise scale, the pairing and the $\frac{1}{2}$ factor.

Pilot in pure bfloat16. An earlier run used pure bfloat16 with round-to-nearest updates, no matched control, Qwen2.5-0.5B and SmolLM-360M, and a 200-step budget. Its results are summarised at the end of Section 4.2; all per-seed data are released.

4.2 Results

The split is function-preserving, and the anti-symmetric pairing cancels drift. In float32, validation perplexity immediately after the DPS split differs from the seed's by at most 1.3×10^{-6} across all runs, the same as exact duplication (1.8×10^{-6}, from re-associating floating-point sums), whereas Gaussian noise of the same RMS shifts it by up to 2.1×10^{-4}. The ablation (Table 5) shows the same ordering at every noise scale: at $C{=}10^{-1}$ the DPS drift is 1.3×10^{-3} against 1.5×10^{-2} for Gaussian noise, and the anti-symmetric pairing removes the drift that a symmetric $(+\delta,+\delta)$ pairing leaves (0 vs. 3.0×10^{-5} at $C{=}10^{-4}$), as Proposition 2 predicts. Removing the $\frac{1}{2}$ factor raises the initial perplexity from 5.13 to 14.47. Under the bfloat16 protocol the split is rounded to nearest, which alone shifts perplexity by up to 0.02.

A split behaves like a learning-rate change. In 14 of 16 settings each of the three splits ends within 0.18 points of the matched control (Table 1, Figure 2); the exceptions are the two noisiest settings, Qwen2.5-1.5B and SmolLM2-1.7B on GSM8K, where the splits are within 0.68 points of the control

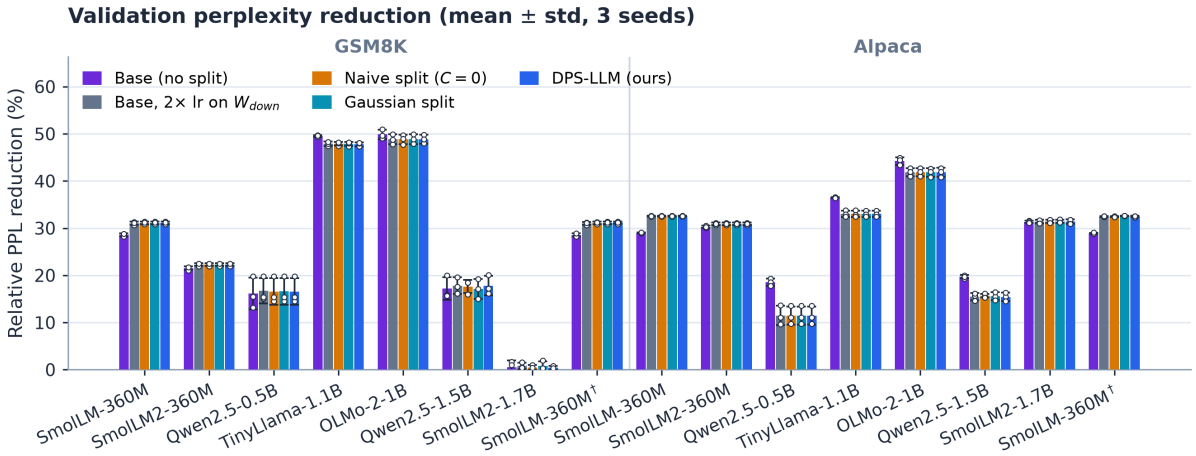


Figure 2: **Benchmark summary**: relative validation-perplexity reduction, mean ± std over three seeds, with individual seeds as dots. In every setting the three splits (orange, teal, blue) match the matched control (grey), whether they are above or below the unexpanded seed (purple). †: bfloat16 with stochastic rounding.

and the seed standard deviation is 0.4–2.4. Only one of sixteen DPS-vs-control comparisons reaches $p < 0.05$ (SmolLM-360M/GSM8K, +0.11 points). Relative to the unexpanded seed at the shared learning rate, splitting looks helpful where the seed is learning-rate-limited (SmolLM-360M: +2.6 on GSM8K, +3.6 on Alpaca) and harmful where a larger step on W_{down} overshoots (Qwen2.5-0.5B/Alpaca: −7.1; Qwen2.5-1.5B/Alpaca: −4.3; TinyLlama: −1.8 and −3.5; OLMo-2-1B: −1.0 and −2.4). The control moves by the same amounts in every case (+2.5, +3.5, −7.1, −4.2, −1.7, −3.4, −1.0, −2.3), so these differences are the step-size effect of Proposition 3, not capacity.

The perturbation does not break the symmetry. In float32 the relative distance between paired neurons after 100 steps equals its value at the split to three significant digits (5.84×10^{-5} for DPS-LLM and 4.13×10^{-5} for Gaussian noise on SmolLM-360M; Figure 3a), and exactly zero for naive duplication. Across the ablation the distance scales linearly with C and never grows, and the final perplexity is identical to four significant digits for DPS-LLM at every $C \leq 10^{-2}$ and for Gaussian noise up to $C{=}10^{-3}$ (Table 5). This is consistent with Proposition 3: a perturbation of relative size C changes the two copies' gradients by $O(C)$, so Adam, whose update depends on the gradient's direction rather than its scale, moves both copies by almost the same step and the initial gap is preserved rather than amplified. Under the bfloat16 protocol, stochastic rounding of each update pulls the copies apart by itself, so even naive duplication reaches a distance of 3.6×10^{-3} on SmolLM-360M (Figure 3b) and $5\text{–}12 \times 10^{-3}$, 64–89% of the DPS-LLM distance, on the four models of 1B parameters and more; this still yields no consistent gain. Of the forty-eight pairwise comparisons among the three splits only one reaches $p < 0.05$ (Gaussian over naive on SmolLM-360M†/Alpaca, +0.21 points), fewer than the 2.4 expected by chance.

At matched learning rates the larger model does not win. Table 3 compares each model's best learning rate for the seed and for DPS-LLM. The differences range from −3.11 to +2.61 points; the seed is ahead on five of eight settings. The two largest differences come from the largest models: on Qwen2.5-1.5B, DPS-LLM peaks at a lower rate than the seed and diverges at 10^{-4} (−15.6% against +3.6% for the seed), as expected if the split doubles the effective step on W_{down}; on SmolLM2-1.7B both peak at the largest rate swept, so the sweep does not bracket its optimum. Within this budget, the 54–75% of added parameters do not translate into a consistently lower perplexity.

Accuracy. GSM8K exact match on 50 problems is a coarse measure (Table 4), but it shows the same pattern. Fine-tuning on reference solutions lowers the accuracy of the two models that start well above zero: Qwen2.5-0.5B from 22% to 9.3% for the seed and 4.7–5.3% for the splits and the control, and OLMo-2-1B from 54% to 16.0% and 6.7–8.7%. In both, the splits fall with the control, consistent with a larger effective step on W_{down} rather than with the split. The two largest models instead improve: Qwen2.5-1.5B from 12% to

Setting	Value
Hardware	Kaggle, 2× NVIDIA T4 (16 GB); runs sequential; layers of the ≥1B models spread over both GPUs
Precision	float32; or bfloat16 weights with stochastic-rounding AdamW (†)
Optimiser	AdamW (Loshchilov and Hutter, 2019), β=(0.9, 0.999), weight decay 0.01
Learning rate	2×10^{-5}, constant, no warm-up (4×10^{-5} on W_{down} for the control)
Budget	100 sequences (batch 1, accumulation 2 ⇒ 50 updates)
Grad. clipping	global norm 1.0
Max seq. length	256 tokens
Trainable	SwiGLU MLP of every layer; rest frozen
Memory	gradient checkpointing (Chen et al., 2016b)
Noise constant	C=10^{-4} (float32), C=10^{-2} (†); C=0 (naive)
Data	256 train / 100 validation sequences; 50 GSM8K test problems for accuracy
Seeds	42, 43, 44 (noise draw, example order); data seed 0
Software	PyTorch (Paszke et al., 2019), Transformers (Wolf et al., 2020), Datasets (Lhoest et al., 2021)

Table 2: Fine-tuning recipe shared by all benchmark runs.

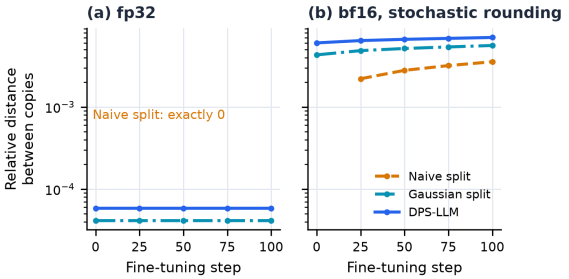


Figure 3: **Distance between the two copies of each split neuron** during fine-tuning of SmolLM-360M, mean over seeds and datasets. In float32 (a) the gap created by the perturbation is preserved but never grows. Under bfloat16 with stochastic rounding (b) the rounding itself separates the copies, including for exact duplication.

Model	Base		DPS-LLM		Δ
	lr	red.	lr	red.	
SmolLM-360M	10^{-4}	34.40	10^{-4}	34.54	+0.14
SmolLM2-360M	5×10^{-5}	23.04	5×10^{-5}	22.83	−0.20
Qwen2.5-0.5B	2×10^{-5}	15.45	10^{-5}	15.07	−0.38
TinyLlama-1.1B†	10^{-5}	51.45	10^{-5}	50.59	−0.87
OLMo-2-1B†	10^{-5}	52.04	10^{-5}	51.83	−0.21
Qwen2.5-1.5B†	5×10^{-5}	20.60	10^{-5}	17.49	−3.11
SmolLM2-1.7B†	10^{-4}	7.37	10^{-4}	9.98	+2.61
SmolLM-360M†	10^{-4}	34.37	10^{-4}	34.57	+0.19

Table 3: **Best learning rate per model** (GSM8K, seed 42; sweep over $\{1, 2, 5, 10\} \times 10^{-5}$): relative perplexity reduction (%) of the seed and of DPS-LLM, each at its own best rate.

14.0–17.3% and SmolLM2-1.7B from 14% to 32.0–34.0%, the latter while its GSM8K perplexity falls by less than 1%, which suggests it mainly learns the answer format; in both, all five conditions lie within 3.3 points, fewer than two of the 50 problems.

Pilot in pure bfloat16. With round-to-nearest bfloat16, the C=10^{-4} perturbation is absorbed by rounding: the two copies of every neuron kept a cosine similarity of 1.0000 for 200 steps, and DPS-LLM, Gaussian noise and duplication were statistically indistinguishable ($p \geq 0.09$). An emulation of bfloat16 rounding on $\mathcal{N}(0, 0.02^2)$ weights shows why: at C=10^{-4} the copies differ in only 0.6% of entries, against 42% at C=10^{-2}. The same run also lost most small updates, since an update of 2×10^{-5} is below half a bfloat16 ulp for most weights; SmolLM-360M's base model improved perplexity by 5.3% in that run despite twice as many steps, against 28.6% in float32 here.

5 Conclusion

The function-preserving half of DPS-LLM works as designed: placing the factor $\frac{1}{2}$ on the down projection grows the SwiGLU blocks of a pre-trained LLM without changing its output, and the antisymmetric, polarity-signed perturbation cancels first-order drift in practice, by two orders of magnitude relative to Gaussian noise. The symmetry-breaking half does not take effect during fine-tuning: the gap between paired neurons is preserved but never amplified, so the split model behaves like its seed, and under Adam that means like its seed with a doubled learning rate on W_{down}. A matched control reproduces every gain and loss we observe, and at each model's best learning rate the split model is not consistently better than the seed. We draw two lessons for work on model growth. Comparisons of a grown model with its seed at a shared learning rate conflate capacity with step size and need the matched control; and symmetry breaking should be verified by measuring whether the copies diverge, not assumed from the perturbation. Whether larger noise, longer training from scratch-scale budgets, or growth during pre-training changes this picture remains open; the

Model	Before	Base	2× ctrl	Naive	Gauss.	DPS
SmolLM-360M	2	0.7	3.3	4.0	4.0	4.0
SmolLM2-360M	0	5.3	7.3	8.0	7.3	8.0
Qwen2.5-0.5B	22	9.3	5.3	4.7	4.7	4.7
TinyLlama-1.1B†	0	2.0	2.7	2.0	4.0	0.7
OLMo-2-1B†	54	16.0	7.3	8.7	6.7	7.3
Qwen2.5-1.5B†	12	14.0	14.7	17.3	16.7	15.3
SmolLM2-1.7B†	14	34.0	33.3	34.0	34.0	32.0
SmolLM-360M†	2	2.7	2.0	3.3	2.0	2.0

Table 4: **GSM8K exact-match accuracy (%)** on 50 test problems before fine-tuning and after it (mean over seeds).

Variant	Init drift	Red. (%)	Pair dist.
Base (no split)	0	22.31	–
Naive split	0	25.78	0
DPS, $C{=}10^{-5}$	0	25.78	$5.8{\times}10^{-6}$
Gaussian, $C{=}10^{-5}$	$+6.2{\times}10^{-7}$	25.78	$4.1{\times}10^{-6}$
DPS, $C{=}10^{-4}$	0	25.78	$5.8{\times}10^{-5}$
Gaussian, $C{=}10^{-4}$	$+3.2{\times}10^{-6}$	25.78	$4.1{\times}10^{-5}$
DPS, $C{=}10^{-3}$	$+5.1{\times}10^{-7}$	25.78	$5.8{\times}10^{-4}$
Gaussian, $C{=}10^{-3}$	$+2.5{\times}10^{-5}$	25.78	$4.1{\times}10^{-4}$
DPS, $C{=}10^{-2}$	$+1.4{\times}10^{-5}$	25.78	$5.8{\times}10^{-3}$
Gaussian, $C{=}10^{-2}$	$+3.5{\times}10^{-4}$	25.75	$4.1{\times}10^{-3}$
DPS, $C{=}10^{-1}$	$+1.3{\times}10^{-3}$	25.80	$5.8{\times}10^{-2}$
Gaussian, $C{=}10^{-1}$	$+1.5{\times}10^{-2}$	25.55	$4.1{\times}10^{-2}$
DPS, symmetric $(+\delta, +\delta)$	$+3.0{\times}10^{-5}$	25.78	0
DPS, no $\frac{1}{2}$ factor	+9.34	60.52	$5.8{\times}10^{-5}$

Table 5: **Ablations** on SmolLM-360M/GSM8K, float32, seed 42, 50 steps. *Init drift*: change of validation perplexity caused by the split (base: 5.13). *Red.*: relative perplexity reduction after fine-tuning. *Pair dist.*: relative distance between paired neurons after fine-tuning.

same operator applies to MoE upcycling and LoRA rank growth, which we have not evaluated.

Limitations

Scale. We evaluate seven models up to 1.7B parameters, fine-tuned for 50 optimiser updates on 256 examples; capacity effects may only appear with larger models, larger datasets or longer training, and in pre-training rather than fine-tuning. **Statistics.** Three seeds per condition detect only differences of a few tenths of a point; the learning-rate sweep and the ablations use a single seed, and the sweep does not bracket the best rate of SmolLM2-1.7B. **Precision.** The four models of 1B parameters and more use bfloat16 with stochastic rounding and a larger noise scale than the float32 models, so their noise condition is not identical; on SmolLM-360M the two protocols agree within 0.2 points for every condition. **Scope.** Only width splitting of the MLP was evaluated, with only the MLP parameters trainable; depth growth, attention-head splitting, the MoE and LoRA regimes, and full-parameter fine-tuning are untested. We measure perplexity and GSM8K exact match but not instruction-following quality for Alpaca-Cleaned. **Data provenance.** The Alpaca-Cleaned validation examples come from its training split; they are unseen during our fine-tuning but may have been seen during the checkpoints' original training.

Ethics Statement

This work modifies the architecture of publicly released language models and uses two public datasets (GSM8K, MIT licence; Alpaca-Cleaned, CC BY-NC 4.0) and seven public checkpoints (Qwen2.5-0.5B-Instruct, Qwen2.5-1.5B-Instruct, SmolLM-360M-Instruct, SmolLM2-360M-Instruct, SmolLM2-1.7B-Instruct, TinyLlama-1.1B-Chat and OLMo-2-0425-1B-Instruct, all Apache 2.0). Growing a model does not change the biases or safety properties of its seed, and a grown model inherits any limitations of the original checkpoint; practitioners should re-run their safety evaluations after expansion. All experiments ran on a free cloud GPU tier with a small compute footprint (about 20 hours on a two-T4 instance for the benchmark, sweeps and ablations, and 4 for the pilot). No human subjects or personal data were involved.

References

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, et al. 2025. SmolLM2: When smol goes big – data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*.

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Leandro von Werra, and Thomas Wolf. 2024. SmolLM: Blazing fast and remarkably powerful. Hugging Face Blog, https://huggingface.co/blog/smollm.

Cheng Chen, Yichun Yin, Lifeng Shang, Xin Jiang, Yujia Qin, Fengyu Wang, Zhi Wang, Xiao Chen, Zhiyuan Liu, and Qun Liu. 2022. bert2BERT: Towards reusable pretrained language models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2134–2148.

Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. 2016a. Net2Net: Accelerating learning via knowledge transfer. In *International Conference on Learning Representations (ICLR)*.

Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016b. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Stefan Elfwing, Eiji Uchibe, and Kenji Doya. 2018. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11.

Utku Evci, Bart van Merriënboer, Thomas Unterthiner, Max Vladymyrov, and Fabian Pedregosa. 2022. GradMax: Growing neural networks using gradient information. In *International Conference on Learning Representations (ICLR)*.

William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.

Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256.

Linyuan Gong, Di He, Zhuohan Li, Tao Qin, Liwei Wang, and Tie-Yan Liu. 2019. Efficient training of BERT by progressively stacking. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 2337–2346.

Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. 2015. Deep learning with limited numerical precision. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 1737–1746.

Ethan He, Abhinav Khattar, Ryan Prenger, Vijay Korthikanti, Zijie Yan, Tong Liu, Shiqing Fan, Ashwath Aithal, Mohammad Shoeybi, and Bryan Catanzaro. 2024. Upcycling large language models into mixture of experts. *arXiv preprint arXiv:2410.07524*.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034.

Dan Hendrycks and Kevin Gimpel. 2016. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Muhammad Irfan. 2021. Discriminative splitting of LSTM-RNN deep neural networks for robust speech recognition. Master's thesis, School of Electrical Engineering and Computer Science (SEECS), National University of Sciences and Technology (NUST), Islamabad, Pakistan.

Fred Jelinek, Robert L Mercer, Lalit R Bahl, and James K Baker. 1977. Perplexity—a measure of the difficulty of speech recognition tasks. *The Journal of the Acoustical Society of America*, 62(S1):S63–S63.

Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088*.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

Dahyun Kim, Chanjun Park, Sanghoon Kim, Wonsung Lee, Wonho Song, Yunsu Kim, Hyeonwoo Kim, Yungi Kim, Hyeonju Kim, Jihoo Kim, et al. 2024. SOLAR 10.7B: Scaling large language models with simple yet effective depth up-scaling. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 23–35.

Diederik P Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*.

Aran Komatsuzaki, Joan Puigcerver, James Lee-Thorp, Carlos Riquelme Ruiz, Basil Mustafa, Joshua Ainslie, Neil Houlsby, and Mostafa Dehghani. 2023. Sparse upcycling: Training mixture-of-experts from dense checkpoints. In *International Conference on Learning Representations (ICLR)*.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*.

Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Patrick von Platen, Suraj Patil, Julien Chaumond, Mariama Drame, Julien Plu, Lewis Tunstall, et al. 2021. Datasets: A community library for natural language processing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 175–184.

Qiang Liu, Lemeng Wu, and Dilin Wang. 2019. Splitting steepest descent for growing neural architectures. In *Advances in Neural Information Processing Systems*, volume 32.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32.

Prajit Ramachandran, Barret Zoph, and Quoc V Le. 2017. Searching for activation functions. *arXiv preprint arXiv:1710.05941*.

Mohammad Samragh, Iman Mirzadeh, Keivan Alizadeh Vahid, Fartash Faghri, Minsik Cho, Moin Nabi, Devang Naik, and Mehrdad Farajtabar. 2024. Scaling smart: Accelerating large language model pre-training with small model initialization. *arXiv preprint arXiv:2409.12903*.

Noam Shazeer. 2020. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*.

Sheng Shen, Pete Walsh, Kurt Keutzer, Jesse Dodge, Matthew Peters, and Iz Beltagy. 2022. Staged training for transformer language models. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, pages 19893–19908.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Stanford Alpaca: An instruction-following LLaMA model. GitHub repository, https://github.com/tatsu-lab/stanford_alpaca.

Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, et al. 2024. 2 OLMo 2 furious. *arXiv preprint arXiv:2501.00656*.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Mojtaba Valipour, Mehdi Rezagholizadeh, Ivan Kobyzev, and Ali Ghodsi. 2023. DyLoRA: Parameter-efficient tuning of pre-trained models using dynamic search-free low-rank adaptation. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 3274–3287.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

Peihao Wang, Rameswar Panda, Lucas Torroba Hennigen, Philip Greengard, Leonid Karlinsky, Rogerio Feris, David Daniel Cox, Zhangyang Wang, and Yoon Kim. 2023. Learning to grow pretrained models for efficient transformer training. In *International Conference on Learning Representations (ICLR)*.

Yite Wang, Jiahao Su, Hanlin Lu, Cong Xie, Tianyi Liu, Jianbo Yuan, Haibin Lin, Ruoyu Sun, and Hongxia Yang. 2024. LEMON: Lossless model expansion. In *International Conference on Learning Representations (ICLR)*.

Tao Wei, Changhu Wang, Yong Rui, and Chang Wen Chen. 2016. Network morphism. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 564–572.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

Lemeng Wu, Bo Liu, Peter Stone, and Qiang Liu. 2020. Firefly neural architecture descent: a general approach for growing neural networks. In *Advances in Neural Information Processing Systems*, volume 33.

yahma. 2023. Alpaca-cleaned: A cleaned and curated version of the Stanford Alpaca dataset. Hugging Face Datasets, https://huggingface.co/datasets/yahma/alpaca-cleaned.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Yiqun Yao, Zheng Zhang, Jing Li, and Yequan Wang. 2024. Masked structural growth for 2x faster language model pre-training. In *International Conference on Learning Representations (ICLR)*.

Pedram Zamirai, Jian Zhang, Christopher R. Aberger, and Christopher De Sa. 2020. Revisiting BFloat16 training. *arXiv preprint arXiv:2010.06192*.

Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385*.

Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. 2023. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. In *International Conference on Learning Representations (ICLR)*.

A Additional Growth Regimes

The same rows/columns operator and anti-symmetric noise apply to two further situations. Both are implemented in the released code but were not part of the benchmark in Section 4.1.

Dense-to-MoE upcycling. A dense SwiGLU block is cloned into E experts with independent calculated-noise draws $\Delta^{(e)}$ and combined by a top-k softmax router (Shazeer et al., 2017; Fedus et al., 2022):

$$\mathrm{Expert}_e(x) = \mathrm{SwiGLU}\big(x;\, W + \Delta^{(e)}\big), \quad (8)$$

$$g(x) = \text{Top-}k\big(\mathrm{softmax}(W_r x)\big). \quad (9)$$

With $k = E$ and a uniform router the layer is initially function-preserving; the perturbations give each expert a distinct gradient trajectory from the first step, which is the failure mode that sparse upcycling (Komatsuzaki et al., 2023) must otherwise wait for the router to resolve.

LoRA rank doubling. For a LoRA adapter $\Delta W = BA$ with $A \in \mathbb{R}^{r \times d_{in}}$, $B \in \mathbb{R}^{d_{out} \times r}$ (Hu et al., 2022), the rank is grown $r \to 2r$ by

$$A' = \begin{bmatrix} A \\ A + \Delta_A \end{bmatrix}, \qquad B' = \begin{bmatrix} B & \Delta_B \end{bmatrix}, \quad (10)$$

where Δ_B is calculated noise on a zero matrix, i.e. near zero. Because the new columns of B are (near) zero, $B'A' = BA + \Delta_B(A + \Delta_A) \approx BA$, so the adapted function is preserved while the optimiser gains r additional rank-one directions.

B Prompt Templates

Each example is rendered as plain text and tokenised with the checkpoint's own tokeniser; no chat template is applied.

Dataset	Template
GSM8K	`Question: {question}\n Answer: {answer}`
Alpaca	`Instruction: {instruction}\n [Input: {input}\n] Response: {output}`

The bracketed `Input` line is emitted only when the example has a non-empty input field.

C Reproducibility

The split operator (`split_huggingface_llm.py`), the synthetic experiment behind Figure 1, the Kaggle notebooks that produced every table and figure, the scripts that generate them from the raw results, and every per-seed result, loss history and neuron-pair distance trace are available at `https://anonymous.4open.science/r/dps-llm`. The benchmark notebook runs on a free Kaggle session with two T4 GPUs; the float32 models take about 0.8 hours each and TinyLlama 2.4 hours for 30 runs, and no run starts after 11 hours, so a session always saves its checkpointed results.